

BPG-03 · BEST PRACTICE GUIDES

Cost breakdown structure and the code of accounts

The coding decision that has to be made before the first commitment is raised

VERSION 1.0 · DRAFT
2026-08-04

PROJECT CONTROLS INSTITUTE GLOBAL, INC.
AI proposes. The professional disposes.

— Summary

How to design the cost breakdown structure and the code of accounts that every project transaction will carry: the difference between the scope axis and the cost axis, cost types and resource classes, segment design and the valid-combination matrix, how the structure maps to the general ledger without either side distorting the other, and why the decision cannot wait. Includes a worked comparison of designing the structure up front against retrofitting it eight months in, with every assumption named.

— About this document

Document	BPG-03
Series	Best Practice Guides
Version	1.0
Status	draft
Date	2026-08-04
Unresolved placeholders	0

Notice. This document is an educational publication of Project Controls Institute Global, Inc., and does not constitute accounting, tax, legal, financial or other professional advice. It must not be relied upon as such. Where a topic depends on the contract or the governing law, entitlement and treatment vary by jurisdiction; take local advice.

Standards. Standards and frameworks are referred to by name and described in this publication's own words. No standard's text, tables, diagrams or question banks are reproduced. All trademarks remain the property of their respective owners, and no endorsement by any standards body, employer or vendor is implied or should be inferred.

Status. Project Controls Institute Global, Inc. is a Delaware Non-Stock Corporation. The Institute has been developed with reference to ISO/IEC 17024 personnel-certification principles and is **not accredited** by ANAB, IAS or any other accreditation body. It intends to seek 501(c)(3) recognition, which has not been granted. No governmental approval, academic equivalence, or guaranteed employment, promotion or salary outcome is claimed or implied.

Examples. All worked examples are illustrative and fictional. Figures are chosen to demonstrate a method, not to describe any real project, organisation or market. Sample examination items, where they appear, are study material maintained separately from any live examination bank.

Completeness. Passages marked **[CONFIRM: ...]** are operational or market facts that have not yet been confirmed from a cited source. A document carrying them is a working draft and is not approved for publication.

© 2026 Project Controls Institute Global, Inc. · All rights reserved.

Contents

1. Two questions, two axes	4
2. Cost types	4
3. Resource classes	5
4. Designing the code	6
5. Mapping to the general ledger	6
6. Why the decision cannot wait	7
7. Direct, indirect and the allocation question	7
8. How this goes wrong	8
9. Worked example	8
9.1 The situation	8
9.2 The retrofit	9
9.3 The up-front alternative	9
9.4 The comparison	10
10. Checklist	10
Related	11
Sources and standards	11
Status and version	12

Cost breakdown structure and the code of accounts

change your mind later.

In one paragraph. How to design the cost breakdown structure and the code of accounts that every project transaction will carry: the difference between the scope axis and the cost axis, cost types and resource classes, segment design and the valid-combination matrix, how the structure maps to the general ledger without either side distorting the other, and why the decision cannot wait. Includes a worked comparison of designing the structure up front against retrofitting it eight months in, with every assumption named.

Who this is for. Cost engineers and cost managers designing or inheriting a coding structure; finance business partners who own the general ledger the project has to reconcile to; project managers about to approve the first purchase orders on a new project.

— 1. Two questions, two axes

Every project cost report is answering one of two questions, and they need different structures.

What did the aeration basins cost? is a **scope** question. It is answered by the work breakdown structure (WBS), and [BPG-02 – The work breakdown structure](#) covers how to build one.

What did we spend on subcontract labour, across the whole project? is a **cost type** question. No amount of scope decomposition answers it, because subcontract labour appears in every branch. It is answered by the **cost breakdown structure (CBS)** — the decomposition of cost by element or type rather than by deliverable.

The two axes are independent, and a single transaction carries a position on both. An invoice for concrete supplied to the inlet works is coded to WBS element 1.3.2 and to cost type "permanent materials". Crossing the axes gives a matrix in which every cell is meaningful: the material cost of the inlet works, the total material cost of the project, and the total cost of the inlet works are all sums along a row, a column or both.

The **code of accounts** is the notation carrying both axes, plus whatever else the project needs to distinguish, on every transaction. Once a transaction is stamped with it, the stamp is what everything downstream sees.

— 2. Cost types

The cost type axis classifies cost by what was bought, and its top level is remarkably stable across sectors. A workable default:

Cost type	What sits in it	Why it is separated
Direct labour	Own-workforce hours charged to project scope	Hours are measurable independently of money; productivity analysis needs them
Subcontract	Work bought as a package, priced by the subcontractor	Commitment behaviour and accrual timing differ from labour
Permanent materials	Materials incorporated into the deliverable	Quantity-controllable; recoverable through valuation
Consumables	Materials consumed in the process	Not quantity-controllable against the deliverable
Plant and equipment	Owned or hired plant, and its running cost	Hire versus own is a distinct commercial decision
Temporary works	Scaffolding, formwork, site facilities	Real cost, no permanent deliverable to attach it to
Freight, duties and logistics	Movement of goods to site	Often the largest surprise on international scope
Professional services	Design, survey, inspection, specialist advice	Bought differently and accrued differently
Indirects and site overhead	Supervision, site establishment, welfare	Must be visible; must not be spread invisibly
Escalation and currency provision	Explicitly held allowances	Must never be mixed into base cost

Two design rules matter more than the list itself. **Direct labour is always split out, and always carries hours as well as money**, because hours are the earliest and most reliable productivity signal on labour-driven work and they are lost the moment labour is bought as a lump sum. And **allowances are never mixed into base cost**, because an escalation allowance buried inside a materials line cannot be released, tracked or reported, and it will be spent on something else.

Contingency and management reserve are held separately again — they are not cost types but reserves, and [BPG-10 – Contingency and management reserve](#) covers their treatment.

— 3. Resource classes

The resource class axis sits beneath cost type and answers what kind of resource. Within direct labour: welder, pipefitter, electrician, supervisor. Within plant: crane, excavator, generator.

It is optional and should be added only where a rate-and-quantity analysis will genuinely be performed, because it multiplies both the code space and the discipline needed to keep it clean. The test: if nobody will ever ask "how many welder-hours at what rate?", do not create a welder resource class. An unused segment does not stay empty — it fills with whatever appeared first in the drop-down list, and that is worse than nothing, because somebody will eventually report from it. Where resource classes are used, define them as rigorously as cost types: a definition, a unit of measure, and what does not belong in them.

— 4. Designing the code

A code of accounts is a concatenation of segments, each of which answers one question. A workable structure for a mid-sized project:

Segment	Length	Answers	Example
Project	2	Which project or contract	W4
WBS element	5	Which scope element	13302
Cost type	3	What was bought	SUB
Resource class	2	What kind of resource	CV
Transaction state	1	Commitment, accrual or actual	A

Four rules govern segment design.

One question per segment. A segment that sometimes means discipline and sometimes means phase is not a segment; it is two segments sharing a field, and every report built on it will need an exception list.

Fixed length, no intelligence beyond position. Variable-length segments break every string operation downstream. Embedding meaning in a digit — "the third character is 7 for offshore scope" — creates a rule that lives nowhere except in the head of the person who invented it.

Room to grow, but not much. A five-character WBS segment supports far more elements than any project needs; a three-character one runs out during execution, and running out means either overloading an existing code or extending the field, both of which break history.

A valid-combination matrix. The theoretical code space is the product of the segments, and most of it is nonsense. With 120 codeable WBS elements, 18 cost types and 6 resource classes, the space is $120 \times 18 \times 6 = 12,960$ combinations. Perhaps 1,400 of those are legitimate on any given project. The matrix that says which are permitted is the difference between a coding structure and a coding suggestion — without it, a requisition can be coded to "temporary works" against a design element, and nothing will stop it.

Name the owner of the code list, and make creation of a new code an authorised act. A code created ad hoc to get a purchase order through approval is the mechanism by which every structure decays.

— 5. Mapping to the general ledger

The project code of accounts and the general ledger chart of accounts are different instruments with different owners and different purposes, and forcing either to be the other produces a structure that serves neither.

The general ledger exists to produce financial statements. Its classifications follow financial reporting: what is capitalised and what is expensed, which categories must be disclosed separately, which period. It is owned by finance, subject to audit, and it changes rarely. The project code of accounts exists to produce management control. Its classifications follow forecasting and accountability: which scope, whose responsibility, what kind of resource, which state of commitment. It is

owned by the project and may legitimately be more granular than the ledger by an order of magnitude.

The correct relationship is a **defined many-to-one mapping** from project codes to ledger accounts, maintained as a controlled document, with two properties: every project code maps to exactly one ledger account, and the mapping is reconciled at every period close. Many project codes mapping to one ledger account is normal and healthy. One project code mapping to two ledger accounts is a defect — it means the code is carrying a distinction the ledger needs but the code does not express, and the split will be made by hand, differently, every month.

Two cautions. The treatment of what may be capitalised, how borrowing costs are handled, and when revenue may be recognised varies by reporting framework and by jurisdiction, and none of it should be assumed from one country's practice; the project's coding structure should carry enough information for finance to apply whatever framework governs, rather than pre-judging it. And where the project and the ledger disagree at close, the reconciliation is the deliverable — an unexplained difference between the cost report and the ledger is a finding, not a rounding.

— 6. Why the decision cannot wait

A purchase order is not a document; it is the head of a stream. Once raised, it generates goods receipts, invoices, accruals, retentions, variations and payment records, and every one of them inherits the code the purchase order carried. Change the code afterwards and you are not editing a field — you are re-coding a stream, in a system that usually requires a formal amendment to do it.

That is why the coding structure sits early in the dependency chain in [BPG-01 – Building a project controls function from zero](#), before tooling and before any reporting. The cost of the decision is roughly constant whenever it is made; the cost of reversing it grows with the transactions already posted and the commitments already open, and \$9 puts numbers on both sides.

Where the structure genuinely cannot be finalised early, there is a legitimate middle path: fix the segment layout and the cost-type list, which are stable, and populate the WBS segment's lower levels as scope is defined. What must not be deferred is the shape, because a shape change is what forces the re-code.

— 7. Direct, indirect and the allocation question

Some cost cannot be attributed to a single scope element without a rule: site supervision, site establishment, shared plant, quality assurance. There are three honest treatments and one dishonest one.

Charge it to a dedicated indirect element. Simple and transparent, and it keeps the deliverable accounts clean — but no deliverable then carries its full cost, so benchmarking against a fully-loaded rate needs a further step.

Allocate it on a defined driver. Spread supervision across deliverables in proportion to direct labour hours, or shared plant in proportion to plant-hours booked. Fully-loaded deliverable costs, at the price of a rule that must be documented, applied consistently and understood by every reader.

Do both, in different views — the indirect element for control, allocation only in a clearly labelled fully-loaded view for estimating. This is the Institute's recommended practice where the project can maintain two views without confusing them.

The dishonest treatment is to allocate without documenting the driver, or to change the driver mid-project. Both make period-on-period comparison meaningless, and both are usually discovered when somebody asks why a deliverable's unit rate moved without anything happening to it.

— 8. How this goes wrong

Coding is decided by whoever raises the first requisition. They select the code that gets it approved. Everyone after copies them, because it worked. Six months later the structure is whatever the first requisition happened to need.

The ledger is used as the project code of accounts. The project inherits finance's chart, which has no scope dimension, so cost can be reported by type and by period but never by deliverable. The scope dimension gets rebuilt in a spreadsheet, maintained by one person, reconciled to nothing.

A segment is overloaded. The cost type segment starts carrying discipline for a few elements because there was no discipline segment. Reports built on cost type now need an exception list, and that list is not in the report — it is in the analyst's head.

Escalation is buried in base rates. The estimate applied escalation inside the unit rates rather than as a separate allowance. Two years in, nobody can say how much of the spend was escalation and how much was quantity growth, so the forecast cannot distinguish a price problem from a scope problem.

Codes are created to solve approval problems. A cost that does not fit anywhere gets a new code rather than a decision. The list grows monotonically, half of it is used once, and the hierarchy stops rolling up cleanly.

Hours are lost. Labour is coded to money only, so the productivity signal — earned hours against burned hours — is unavailable, and the project loses its earliest indicator of a labour overrun.

The mapping to the ledger is undocumented. It lives as a formula in a workbook; the person who wrote it moves on; the monthly reconciliation becomes an exercise in re-deriving somebody else's intent, and the difference is written off as timing.

— 9. Worked example

Illustrative figures. All effort is in person-hours; monetary values are in generic currency units. No real project, organisation or system is implied. The figures below are the shape of the decision, not a benchmark — the inputs must be measured on your own project before the answer means anything.

9.1 The situation

A project is eight months into execution. The coding structure was deferred at start-up: a provisional two-segment code (project and a rough scope grouping) has been used, with no cost type and no resource class. A proper structure is now needed, because the client has asked for cost by cost type and the forecast cannot separate a price problem from a quantity problem.

Volumes at the point of decision.

Item	Volume
Cost transactions posted (invoice lines, timesheet lines, journals)	4,200
Open commitments requiring a coding amendment	190
Monthly cost reports already issued	12

9.2 The retrofit

Assumptions. 70 % of transactions map mechanically by a lookup rule on the old segment; the remaining 30 % need a human decision. A manual decision takes 4 minutes including the check. A purchase order coding amendment takes 25 minutes including supplier notification and internal approval. Restating an issued report so the trend is continuous takes 3 hours each. Two reconciliation cycles at the new code level are needed before the ledger and the cost report agree, at 16 hours each.

Task	Calculation	Hours
Build and test the crosswalk	Fixed effort	40.0
Manual re-coding	$4,200 \times 0.30 = 1,260 \text{ items} \times 4 \text{ min} = 5,040 \text{ min}$	84.0
Commitment amendments	$190 \times 25 \text{ min} = 4,750 \text{ min}$	79.2
Restating issued reports	$12 \times 3 \text{ h}$	36.0
Reconciliation to the ledger	$2 \times 16 \text{ h}$	32.0
Subtotal		271.2
Contingency at 20 %	271.2×0.20	54.2
Total		325.4

Checks: $4,200 \times 0.30 = 1,260$; $1,260 \times 4 = 5,040$ minutes; $5,040 \div 60 = 84.0$ hours. $190 \times 25 = 4,750$ minutes; $4,750 \div 60 = 79.17$, rounded to 79.2 hours. Subtotal $40.0 + 84.0 + 79.2 + 36.0 + 32.0 = 271.2$. Contingency $271.2 \times 0.20 = 54.24$, rounded to 54.2. Total $271.2 + 54.2 = 325.4$ hours.

Elapsed time. Assigning one person at 32 productive hours a week to the exercise: $325.4 \div 32 = 10.2$ weeks. That is roughly two and a half reporting cycles during which the comparative trend is unavailable, because the old codes and the new codes describe different things.

9.3 The up-front alternative

Assumptions. A two-day design workshop with three people — cost engineering, finance and procurement — at 7 productive hours a day. Configuration of the code list and the valid-combination matrix in the cost system, plus the mapping document to the ledger: 24 hours. Test postings and sign-off: 8 hours.

Task	Calculation	Hours
Design workshop	3 people × 2 days × 7 h	42.0
Configuration and mapping document	Fixed effort	24.0
Test postings and sign-off	Fixed effort	8.0
Total		74.0

Check: $42.0 + 24.0 + 8.0 = 74.0$ hours.

9.4 The comparison

```
Ratio = retrofit effort ÷ up-front effort
      = 325.4 ÷ 74.0
      = 4.4
```

The retrofit costs about **4.4 times** the up-front design in direct effort. Expressed as an addition rather than a ratio, the avoidable cost is $325.4 - 74.0 = \mathbf{251.4 \text{ hours}}$ — because the up-front work has to be done either way; deferring it does not remove it, it only adds the re-coding to it.

What this does not include, and it matters more than what it does. The 10.2 weeks of broken comparative reporting has no line in the table, and on most projects it is the larger cost: for two and a half cycles the project cannot answer "is this getting better or worse?", which is the only question the cost report exists to answer. Nor does the table include the decisions taken during those cycles on a cost report that cannot separate price from quantity.

Sensitivity. The dominant inputs are the transaction volume and the manual proportion. At 8,000 transactions rather than 4,200, with everything else unchanged, the manual re-coding becomes $8,000 \times 0.30 = 2,400 \text{ items} \times 4 \text{ minutes} = 9,600 \text{ minutes} = 160.0 \text{ hours}$, and the subtotal becomes $40.0 + 160.0 + 79.2 + 36.0 + 32.0 = 347.2 \text{ hours}$, giving a total of $347.2 \times 1.20 = \mathbf{416.6 \text{ hours}}$ and a ratio of $416.6 \div 74.0 = \mathbf{5.6}$. The lesson is directional and robust: the retrofit cost scales with elapsed time, and the up-front cost does not.

10. Checklist

Take this into the coding design workshop, or into the review of a structure you have inherited.

Design

- [] Does every segment answer exactly one question, and is that question written down?
- [] Are all segments fixed-length, with no meaning embedded in individual characters?
- [] Is direct labour separated, and does it carry hours as well as money?
- [] Are escalation, currency provision and any other allowance held outside base cost?
- [] Does a resource class exist that nobody will ever report from? (If so, delete it now.)
- [] Is there a valid-combination matrix, and does the system enforce it?

Ownership

- [] Who may authorise a new code, and how long does it take? (If it is faster to invent one, they will.)

- [] Is the code list a controlled document with a version and an owner?
- [] Are retired codes retired permanently, never reissued?

The ledger

- [] Does every project code map to exactly one general ledger account?
- [] Is the mapping a controlled document rather than a workbook formula?
- [] Is it reconciled every period, with differences explained rather than written off as timing?
- [] Has finance confirmed the structure carries enough information for their reporting framework, without the project pre-judging the treatment?

Indirects

- [] Is every indirect cost either charged to a named indirect element or allocated on a documented driver?
- [] If costs are allocated, is the driver written down, and has it changed during the project?
- [] Is there a clearly labelled distinction between the control view and any fully-loaded view?

If you are considering deferring the decision

- [] How many transactions are currently posting each month?
- [] How many commitments are open, and what does an amendment cost in your system?
- [] Run §9's arithmetic with your own numbers, and take the answer to the person asking for the delay.

— Related

- [BPG-01 – Building a project controls function from zero](#) — where the coding decision sits in the sequence, and what else it blocks.
- [BPG-02 – The work breakdown structure](#) — the scope axis this structure crosses.
- [BPG-07 – Accruals and cut-off discipline](#) — the transaction-state segment in practice, and why commitment, accrual and actual must be distinguishable.
- [BPG-11 – Change orders and variations](#) — how new scope acquires a code without breaking the structure.
- [TPL-03 – Cost breakdown structure and code of accounts](#) — the instrument, with segment definitions and a valid-combination matrix.
- [TPL-02 – Work breakdown structure and WBS dictionary](#) — the companion template for the scope axis.

— Sources and standards

Drawn from the Institute's Body of Knowledge: Domain 5 (Cost Management and Cost Control) for the cost breakdown structure and the commitment-accrual-actual states, Domain 1 (Foundations of Accounting for Project Controls) for cost coding and the relationship to the general ledger, and Domain 11 (Business Process Cycles) for the procure-to-pay controls that the coding structure depends on.

The retrofit arithmetic in §9 is constructed for this guide from stated assumptions. It is not survey data and not a benchmark; the inputs must be measured on your own project. Where financial reporting frameworks govern capitalisation, revenue recognition or disclosure, the treatment varies by framework and by jurisdiction, and none is described here as universal. No external standard, chart of accounts or table is reproduced.

— Status and version

Founding-stage document · Version 1.0 — effective date to be confirmed · Reviewed under PCI governance. PCI makes no claims of accreditation or recognition beyond what is true today.